ANDROID DEVELOPMENT GUIDE

Android Layouts

A visual guide to LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout, TableLayout & GridLayout

What's inside
Structure, XML syntax, and a visual diagram for each layout type

Also covers
Performance notes, common pitfalls, and modern alternatives

Reference tables
Side-by-side comparison and "which layout for which screen"

Level
Beginner to intermediate Android developers

Covers structure, XML syntax, visual arrangement, performance notes, and when to use each layout in modern Android apps.

OVERVIEW

What Is an Android Layout?

A layout defines **how UI components** — TextView, Button, ImageView, EditText, and similar widgets — are positioned and sized on the screen. Every layout is itself a `ViewGroup`, a special view that can contain other views ("children") and is responsible for measuring and arranging them. Choosing the right layout affects rendering performance, how easy the UI is to maintain, and how well it adapts to different screen sizes.

> **Good to know:** Since 2020, Google's recommended approach for new UI code is **Jetpack Compose**, a declarative Kotlin toolkit that replaces XML layouts entirely. The layouts covered in this guide (the "View system") remain the foundation of most existing Android codebases and are essential to understand when maintaining or extending older apps.

1. LinearLayout

Stacks children in one direction — vertical or horizontal. Simple and beginner-friendly.

2. RelativeLayout

Positions children relative to each other or the parent. Mostly superseded today.

3. ConstraintLayout

Positions children using constraints solved as one flat system. The modern standard.

4. FrameLayout

Stacks children on top of one another in z-order. Great for fragments and overlays.

5. TableLayout

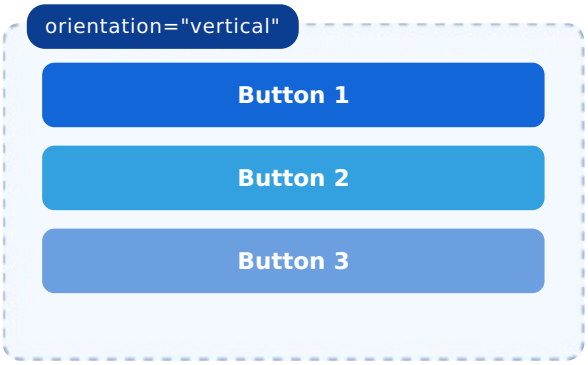
Arranges children into rows and columns, spreadsheet-style. Good for structured forms.

6. GridLayout

Places children into a fixed rectangular grid. Ideal for calculators and dashboards.

1 LinearLayout

Arranges every child in a **single direction** — either `vertical` or `horizontal` — one after another, in the order they appear in the XML.




```xml
<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">

  <Button android:text="Button 1"/>
  <Button android:text="Button 2"/>
  <Button android:text="Button 3"/>

</LinearLayout>
```

> **Important detail:** `layout_weight` only distributes *remaining* space after every view's declared size has been measured. Setting a child's size to `0dp` on the weighted axis makes weight distribution accurate and is the standard pattern for equally-sized children.

Advantages

- Simple and beginner-friendly
- Good fit for forms and menus
- Weights split space proportionally

Disadvantages

- Deep nesting hurts performance
- Awkward for non-linear screens

2 RelativeLayout LEGACY

Positions each child **relative to sibling views or to the parent** — e.g. below a view, to its right, or aligned to the parent's edges.




```xml
<RelativeLayout>
  <TextView
    android:id="@+id/title"
    android:text="Title"/>
  <Button
    android:layout_below="@id/title"
    android:text="Login"/>
</RelativeLayout>
```

Why it fell out of favor: RelativeLayout typically needs **two measure/layout passes** to resolve relative positions, which is slower than ConstraintLayout's single-pass constraint solver. Google now recommends ConstraintLayout for anything beyond the simplest case.

Advantages

- Avoids deep nesting for moderate UIs
- Familiar attribute-based syntax

Disadvantages

- XML gets hard to read as rules grow
- Slower than ConstraintLayout

3 ConstraintLayout RECOMMENDED

Positions every child using **constraints** resolved together as one system. It has been the default layout generated by Android Studio's UI templates since 2016.




```
<Button
  android:text="Login"
  app:layout_constraintTop_toTopOf="parent"
  app:layout_constraintStart_toStartOf="parent"
  app:layout_constraintEnd_toEndOf="parent"
  app:layout_constraintBottom_toBottomOf="parent"/>
```

Beyond the basics: ConstraintLayout also supports **chains**, **guidelines** and **barriers**, and **dimension ratios** — features that let it replace most LinearLayout, RelativeLayout, and even some GridLayout use cases in a single flat view hierarchy.

Advantages

- Best performance, flat hierarchy
- Chains, ratios, guidelines, barriers

Disadvantages

- Steeper learning curve
- Verbose XML without a visual editor

4 FrameLayout

Stacks children **directly on top of each other**. Later children in the XML are drawn **above** earlier ones, so it's typically used to show one view at a time or to overlay content.




```
<FrameLayout>
  <ImageView/>
  <TextView
    android:text="Welcome"/>
</FrameLayout>
```

> **Common real-world use:** FrameLayout is the standard container for hosting a single `Fragment` at a time (via `FragmentContainerView`), and is used for caption overlays on images/video and simple loading-spinner overlays.

Advantages

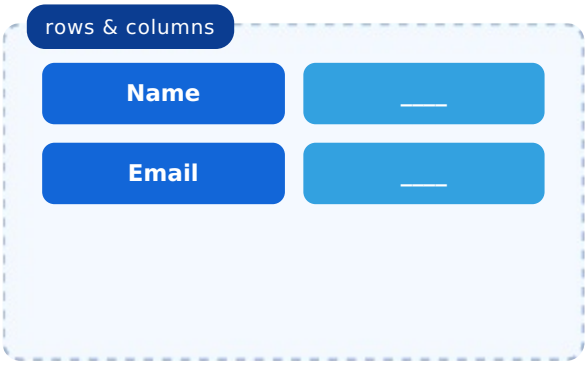
- Extremely lightweight and simple
- Ideal fragment container

Disadvantages

- Not meant for many separate views
- No relative positioning between children

5 TableLayout LEGACY

Arranges children into **rows and columns**. Each row is declared with a `<TableRow>`; column widths auto-size to fit the widest cell in that column across all rows.




```
<TableLayout>
  <TableRow>
    <TextView android:text="Name"/>
    <EditText/>
  </TableRow>
</TableLayout>
```

> **Useful attributes:** `android:stretchColumns` and `android:shrinkColumns` control whether specific columns expand to fill extra space or compress to fit — without them, columns simply hug their content.

Advantages

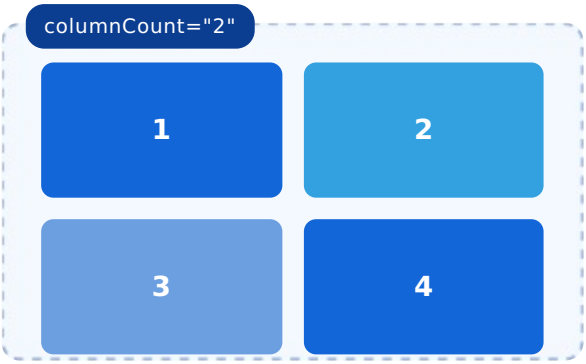
- Intuitive for spreadsheet-like data
- Good for simple data-entry forms

Disadvantages

- Less flexible than ConstraintLayout
- No built-in cell spanning

6 GridLayout

Places children into a **rectangular grid** controlled with `rowCount` / `columnCount`. Introduced in **API level 14 (Android 4.0)** to give grid arrangement without TableLayout's row-based structure.




```
<GridLayout
  android:columnCount="2">
  <Button android:text="1"/>
  <Button android:text="2"/>
  <Button android:text="3"/>
  <Button android:text="4"/>
</GridLayout>
```

Cell spanning: A single child can span multiple rows or columns using `layout_rowSpan` / `layout_columnSpan`, letting GridLayout build dashboard-style or calculator-style UIs that plain TableLayout cannot do as cleanly.

Advantages

- Purpose-built for calculators, dashboards
- Supports row/column spanning

Disadvantages

- Limited for irregular positioning
- Largely superseded by ConstraintLayout

REFERENCE

Side-by-Side Comparison

Layout	Arrangement	Best Use	Advantages	Disadvantages
LinearLayout	Horizontal or vertical stack	Forms, simple screens	Easy to use, supports weights	Deep nesting hurts performance
RelativeLayout	Relative to parent/siblings	Medium-complexity screens	Flexible positioning	Two measure passes; mostly replaced

ConstraintLayout	Constraint-solver based	Modern app interfaces	High performance, chains/ratios/barriers	Learning curve for beginners
FrameLayout	Stacked (z-order)	Fragments, overlays	Lightweight and simple	Not for many arranged views
TableLayout	Rows and columns	Data-entry forms	Structured, spreadsheet-like	No row spanning; less flexible
GridLayout	Fixed grid (rows × cols)	Calculators, dashboards	Row/column spanning support	Limited for irregular layouts

When Should You Use Each Layout?

Scenario	Recommended Layout
Login form	LinearLayout or ConstraintLayout
Registration form	ConstraintLayout
Calculator	GridLayout (or ConstraintLayout chains)
Image with text overlay	FrameLayout
Table-like data entry	TableLayout
Modern responsive application	ConstraintLayout
Simple vertical menu	LinearLayout
New project in 2026	Jetpack Compose (declarative UI)

Recommendation: Start with **LinearLayout** to understand basic view arrangement, then move to **ConstraintLayout** — the standard for production Android UIs today. For any brand-new project, also evaluate **Jetpack Compose**, which Google now recommends over the XML View system entirely, since these XML layouts are maintained mainly for existing apps.